



A T M E
College of Engineering

13th KM Stone, Bannur Road, Mysore - 560 028

**Department of CSE–Artificial Intelligence & Machine Learning
(Academic Year 2026-27)**



LABORATORY MANUAL

SUBJECT: COMPUTER NETWORKS LABORATORY

SUB CODE: BCS502

SEMESTER: V

SCHEME: 2022

Prepared By

**Ms. Ruchitha BS
Programmer**

Verified By

**HARSHITHA V
Assistant Professor
Dept. of CSE-AI & ML**

Approved by

**Dr. ANIL KUMAR C.J
Assoc. Professor & Head
Dept. of CS-AI & ML**

Institute Vision

- Development of academically excellent, culturally vibrant, socially responsible and globally competent human resources.

Institute Mission

- To keep pace with advancements in knowledge and make the students competitive and capable at the global level.
- To create an environment for the students to acquire the right physical, intellectual, emotional and moral foundations and shine as torch bearers of tomorrow's society.
- To strive to attain ever-higher benchmarks of educational excellence.

Department Vision

To impart technical education in the field of Artificial intelligence and machine learning of topnotch quality with a high level of professional competence, social obligation, and global cognizance among the students.

Department Mission

- To impart technical education that is up to date, relevant and makes students to compete at global level
- Fostering an ambiance where students can adopt the suitable moral, intellectual, emotional, and physical attributes to shine as the leaders of tomorrow's society.
- To strive to meet ever higher educational standard.

Program Outcomes (PO's)

1. **Engineering knowledge:** Apply fundamental knowledge to solve complex engineering problems.
2. **Problem analysis:** Identify and formulate complex problems to reach substantiated conclusions.
3. **Design/development of solutions:** Create solutions that meet specific needs, considering health, safety, and environmental factors.
4. **Conduct investigations of complex problems:** Use research-based methods to design experiments, analyze, and interpret data.
5. **Engineering Tool Usage:** Select and apply modern techniques and IT tools to engineering activities.
6. **The Engineer and the World:** Understand the impact of professional engineering solutions on society and the environment.
7. **Ethics:** Commit to professional ethics and responsibilities.
8. **Individual and Collaborative Team Work:** Function effectively as an individual or team member in multidisciplinary settings.
9. **Communication:** Communicate effectively regarding complex engineering activities with the engineering community and society.
10. **Project Management and Finance:** Apply management and financial principles to project work.
11. **Life-Long Learning:** Engage in independent and life-long learning in the context of technological change.

Program Educational Objectives (PEO's):

PEO1: Graduates will be able to hone their problem-solving abilities and capacity to offer solutions to challenges that arise in the actual world.

PEO2: Able to design and develop AI based solutions to real-world problems in a business, research, or social environment.

PEO3: Graduates shall acquire and inculcate corporate culture, core attributes, and leadership qualities as well as professional etiquette's and lifelong learning.

Program Specific Outcomes (PSO's)

PSO1: Ability to design and develop artificial intelligent based solutions by applying optimal algorithms to solve real world issues.

PSO2: Ability to apply suitable AI tools and techniques to offer solutions in the various domains of engineering.

COMPUTER NETWORKS		Semester	V
Course Code	BCS502	CIE Marks	50
Teaching Hours/Week (L:T:P: S	3:0:2:0	SEE Marks	50
Total Hours of Pedagogy	40 hours Theory + 8-10 Lab slots	Total Marks	100
Credits	04	Exam Hours	03
Examination nature (SEE)	Theory/practical		

Sl.NO	Experiments
1	Implement three nodes point – to – point network with duplex links between them. Set the queue size, vary the bandwidth, and find the number of packets dropped.
2	Implement transmission of ping messages/trace route over a network topology consisting of 6 nodes and find the number of packets dropped due to congestion.
3	Implement an Ethernet LAN using n nodes and set multiple traffic nodes and plot congestion window for different source / destination.
4	Develop a program for error detecting code using CRC-CCITT (16- bits).
5	Develop a program to implement a sliding window protocol in the data link layer.
6	Develop a program to find the shortest path between vertices using the Bellman-Ford and path vector routing algorithm.
7	Using TCP/IP sockets, write a client – server program to make the client send the file name and to make the server send back the contents of the requested file if present.
8	Develop a program on a datagram socket for client/server to display the messages on client side, typed at the server side.
9	Develop a program for a simple RSA algorithm to encrypt and decrypt the data.
10	Develop a program for congestion control using a leaky bucket algorithm.

Course outcomes (Course Skill Set):

At the end of the course, the student will be able to:

- **Explain** the fundamentals of computer networks.
- **Apply** the concepts of computer networks to demonstrate the working of various layers and protocols in communication network.
- **Analyze** the principles of protocol layering in modern communication systems.
- **Demonstrate** various Routing protocols and their services using tools such as Cisco packet tracer.

PRACTICAL COMPONENT OF IPCC

Assessment Details (both CIE and SEE)

The weightage of Continuous Internal Evaluation (CIE) is 50% and for Semester End Exam (SEE) is 50%. The minimum passing mark for the CIE is 40% of the maximum marks (20 marks out of 50) and for the SEE minimum passing mark is 35% of the maximum marks (18 out of 50 marks). A student shall be deemed to have satisfied the academic requirements and earned the credits allotted to each subject/course if the student secures a minimum of 40% (40 marks out of 100) in the sum total of the CIE (Continuous Internal Evaluation) and SEE (Semester End Examination) taken together.

CIE for the theory component of the IPCC (maximum marks 50)

- IPCC means practical portion integrated with the theory of the course.
- CIE marks for the theory component are **25 marks** and that for the practical component is **25marks**.
- 25 marks for the theory component are split into **15 marks** for two Internal Assessment Tests (Two Tests, each of 15 Marks with 01-hour duration, are to be conducted) and **10 marks** for other assessment methods mentioned in 22OB4.2. The first test at the end of 40 -50% coverage of the syllabus and the second test after covering 85-90% of the syllabus.
- Scaled-down marks of the sum of two tests and other assessment methods will be CIE marks for the theory component of IPCC (that is for **25 marks**).
- The student has to secure 40% of 25 marks to qualify in the CIE of the theory component of IPCC.

CIE for the practical component of the IPCC

- **15 marks** for the conduction of the experiment and preparation of laboratory record, and **10 marks** for the test to be conducted after the completion of all the laboratory sessions.
- On completion of every experiment/program in the laboratory, the students shall be evaluated including viva-voce and marks shall be awarded on the same day.
- The CIE marks awarded in the case of the Practical component shall be based on the continuous evaluation of the laboratory report. Each experiment report can be evaluated for 10 marks. Marks of all experiments' write-ups are added and scaled down to **15 marks**.
- The laboratory test (**duration 02/03 hours**) after completion of all the experiments shall be conducted for 50 marks and scaled down to **10 marks**.
- Scaled-down marks of write-up evaluations and tests added will be CIE marks for the laboratory component of IPCC for **25 marks**.

The student has to secure 40% of 25 marks to qualify in the CIE of the practical component of the IPCC. SEE for IPCC

Sl.NO	EXPERIMENTS	Page No.
1	INTRODUCTION	1
2	Program1: Implement three nodes point – to – point network with duplex links between them. Set the queue size, vary the bandwidth, and find the number of packets dropped.	10
3	Program 2: Implement transmission of ping messages/trace route over a network topology consisting of 6 nodes and find the number of packets dropped due to congestion.	13
4	Program 3: Implement an Ethernet LAN using n nodes and set multiple traffic nodes and plot congestion window for different source / destination.	17
5	Program4 Develop a program for error detecting code using CRC-CCITT (16- bits).	20
6	Program 5: Develop a program to implement a sliding window protocol in the data link layer.	24
7	Program 6: Develop a program to find the shortest path between vertices using the Bellman-Ford and path vector routing algorithm.	26
8	Program 7: Using TCP/IP sockets, write a client – server program to make the client send the file name and to make the server send back the contents of the requested file if present.	29
9	Program 8: Develop a program on a datagram socket for client/server to display the messages on client side, typed at the server side	34
10	Program 9: Develop a program for a simple RSA algorithm to encrypt and decrypt the data.	38
11	Program 10: Develop a program for congestion control using a leaky bucket algorithm.	44
12	Viva Questions and Answers	48

INTRODUCTION

Introduction to NS-2

NS-2 stands for Network Simulator Version 2. It is an open-source event-driven simulator designed specifically for research in computer communication networks. Network Simulator 2 (NS2) is a popular open source network simulator for carrying out network experimentation. Way back when it was being designed, its primary usage was to analyze the performance of congestion control algorithms implemented in Transmission control protocol (TCP). Even today, it remains the most widely used network simulator for TCP research. Over the period of time, it gained wide acceptance in industry, and now supports simulation of latest wired as well as wireless networking protocols (e.g., routing algorithms, TCP, User Data Protocol (UDP)) and paradigms such as Mobile Ad hoc Networks (MANETs) Vehicular Ad hoc Network (VANETs), etc. Another simulator called ns-3 has gained a lot of popularity in the recent past. It is not a sequel of NS-2. NS-3 APIs are not compatible with those of NS-2 API. Both are completely different tools.

Features of NS-2:

- It is a discrete event simulator for networking research.
- It provides substantial support to simulate protocols like TCP, FTP, UDP &DSR.
- It simulates wired and wireless network.
- It is primarily UNIX based.
- Uses TCL as its scripting language.
- Otcl: Object oriented support Tcl
- TclCL: Tcl with Classes and OTcl linkage
- Discrete event scheduler

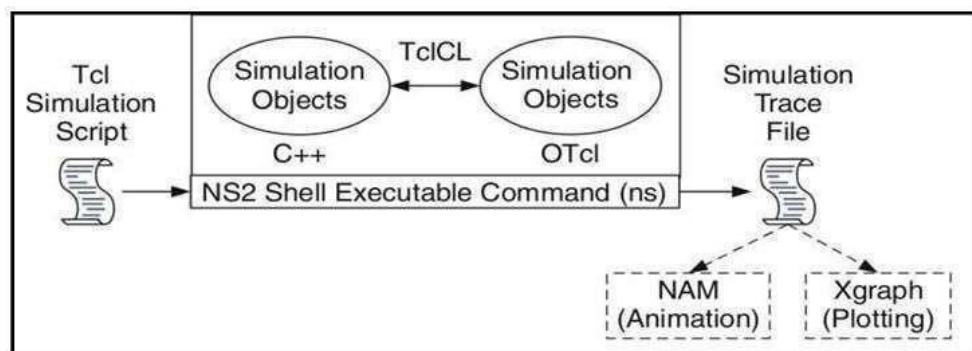


Fig. 1 Basic Architecture of Network Simulator

Why two languages? (TCL and C++)

- NS2 consists of two key languages: C++ and Object-oriented Tool Command Language(OTcl).
- The C++ defines the internal mechanism (i.e., a backend) of the simulation objects.
- The OTcl sets up simulation by configuring the objects as well as scheduling discrete events (i.e., a front end).
- The C++ and the OTcl are linked together using Tcl CL.
- NS2 uses OTcl to create and configure a network, and uses C++ to run simulation.
 - C++ is fast to run but slow to change.
 - OTcl, on the other hand, is slow to run but fast to change.
 - We write a Tcl simulation script and feed it as an input argument to NS2 when running simulation (e.g., executing “nsmyfirst_ns.tcl”).
 - Here, “ns” is a C++ executable file obtained from the compilation.
 - myfirst_ns.tcl is an input configuration file specifying system parameters and configuration such as nodes, link, and how they are connected.
 - C++ is used for the creation of objects because of speed and efficiency.
 - OTcl is used as a front-end to setup the simulator, configure objects and schedule event because of its ease of use.

Tcl scripting

- Tcl is a general purpose scripting language.[Interpreter]
- Tcl runs on most of the platforms such as Unix, Windows, and Mac.
- The strength of Tcl is its simplicity.
- It is not necessary to declare a data type for variable prior to the usage

Structure of NS-2 Program:

- Creating a SimulatorObject
- Setting up files for trace & NAM
- Tracing files using their commands
- Closing trace file and starting NAM
- Creating LINK & NODE topology & Orientation of links

Working of NS-2

- NS2 provides users with executable command ns which takes an input argument, the name of a Tcl simulation scripting file.
- Users are feeding the name of a Tcl simulation script (which sets up a simulation as an input argument of an NS2 executable command ns.
- In most cases, a simulation trace file is created, and is used to plot graph and/or to create animation.

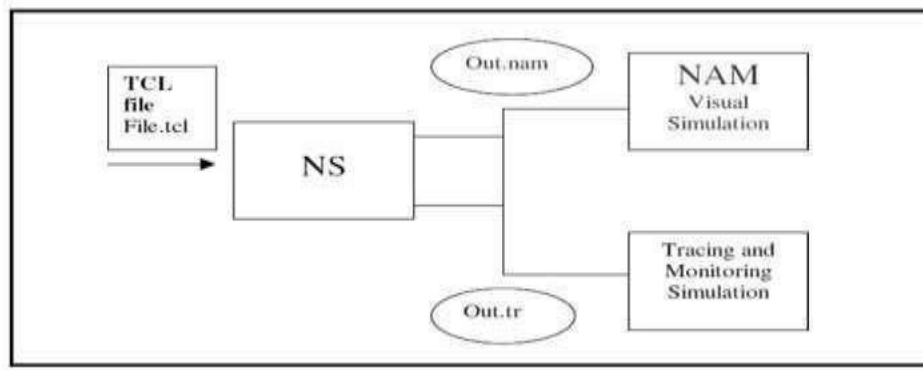


Fig 2. Working of Network Simulator 2

Trace file and NamTrace file:

- Once the simulation is complete, we can see two files: “trace.tr”, and “nam.out”.
- The trace file (trace.tr) is a standard format used byns2.
- In ns2, each time a packet moves from one node to another, or onto a link, or into a buffer, etc., it gets recorded in this trace file.
- Each row represents one of these events and each column has its own meaning.
- Start nam with the command 'nam <nam-file>' where '<nam-file>' is the name of a nam trace file that was generated byns.

Network Animator Window

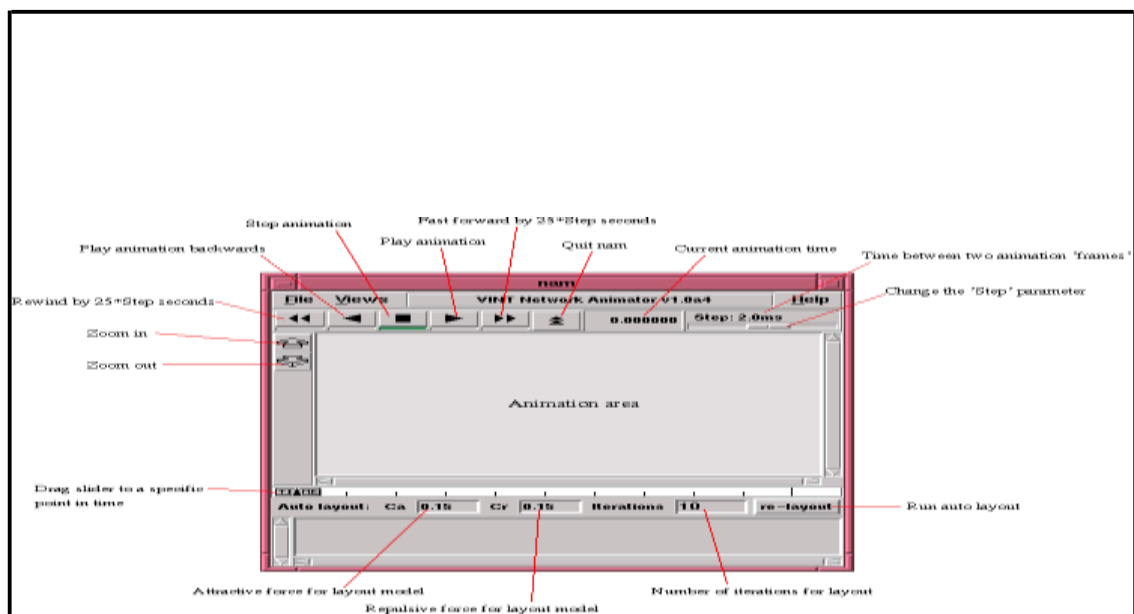


Fig. 3 Details of NAM Window

Advantages and Disadvantages of NS2:**Advantages**

1. Open Source
2. Complex scenarios can be easily tested.
3. Results can be quickly obtained – more ideas can be tested in a smaller timeframe.
4. Supported protocols
5. Supported platforms
6. Modularity

Disadvantages

1. Limitation in designing large scale systems
2. May be slow compared to real time network and computationally expensive
3. Does not reflect reality in large and complex networks.
4. Statistical uncertainty in results

Steps to create a scenario file:

Step1: Declare Simulator and setting output file

Step2: Setting Node and Link

Step3: Setting Agent

Step4: Setting Application

Step5: Setting Simulation time and schedules

Step6: Declare finish.

Step 1: Declare Simulator and setting

\$ns [new Simulator]	#first line of tcl script. Creates ns object
get file [open out.tr w] \$ns trace-all \$file	#open the trace file
get namfile [open out.nam w] \$ns namtrace-all \$namfile	#open the namfile

Step 2: Setting Node and Link

\$ n0 [\$ns node]	setting a node
ns duplex-link \$n0 \$n2 3Mb 5ms DropTail	#bidirectional link between n0 and n2 is declared bandwidth 3Mbps and delay 5ms. DropTail is a waiting queue type.
\$ns duplex-link-op \$n0 \$n2 orient right-down	#Sets positions of node and link for Nam. It does not affect to the result of simulation
\$ns queue-limit \$n2 \$n3 20	#The length of queue on the link from n2 to n3 is 20[packets].
\$ns duplex-link-op \$n2 \$n3 queuePos 0.5	#The position of queue is set for Nam, 0.5 is the angle between link and queue, it equals to (0.5_).

Step 3: Setting Agent

UDP Agent : To use UDP in simulation, the sender sets the Agent as UDP Agent while the receiver sets to Null Agent. Null Agents do nothing except receiving the packets.

set udp [newAgent/UDP] \$ns attach-agent \$n0 \$udp set null [newAgent/Null] \$ns attach-agent \$n3 \$null	#udp and null Agent are set for n0 and n3, respectively
\$ns connect \$udp \$null	Declares the transmission between udp and null.
\$udp set fid_ 0	Sets the number for data flow of udp. This number will be recorded to all packets which are sent from udp
\$ns color 0 blue	Mark the color to discrete packet for showing result on NAM.

TCP Agent: To use TCP in simulation, the sender sets the Agent as TCP Agent while the receiver sets to TCPSink Agent. When receiving a packet, TCPSink Agent will reply an acknowledgment packet (ACK). Setting Agent for TCP is similar to UDP.

set tcp [new Agent/TCP] \$ns attach-agent \$n1 \$tcp set sink [new Agent/TCPSink] \$ns attach-agent \$n3 \$sink	# tcp and sink Agent are set for n1 and n3, respectively
\$ns connect \$tcp \$sink	#declares the transmission between tcp and sink.
\$tcp set fid_ 1	#sets the number for data flow of tcp. This number will be recorded to all packet which are sent from tcp
\$ns color 1 red	#mark the color to discrete packet for showing result on Nam.

Step 4: Setting Application

In general, UDP Agent uses CBR Application while TCP Agent uses FTP Application.

set cbr [new Application/Traffic/CBR] \$cbr attach-agent \$udp
set ftp [new Application/FTP] \$ftp attach-agent \$tcp

Step 5: Setting time schedule for simulation

Time schedule of a simulation is set as below:

\$ns at 1.0 "\$cbrstart" \$ns at 3.5 "\$cbrstop"	cbr transmits data from 1.0[sec] to 3.5[sec]
\$ns at 1.5 "\$ftpstart" \$ns at 3.0 "\$ftpstop"	ftp transmits data from 1.5[sec] to 3.0[sec].

Step 6: Declare finish

After finish setting, declaration of finish is written at the end of file.

\$ns at 4.0 "finish"	The finish function is used to output data file at the end of simulation.
proc finish {} { global ns file namfile tcpfile \$ns flush-trace close \$file close \$namfile close \$tcpfile exit 0 }	

Execute Simulation and start Nam

Byexecuting below command line, simulation will be started and shows the animation of simulation

```
ns sample.tcl
nam out.nam
```

View trace file (out.tr)

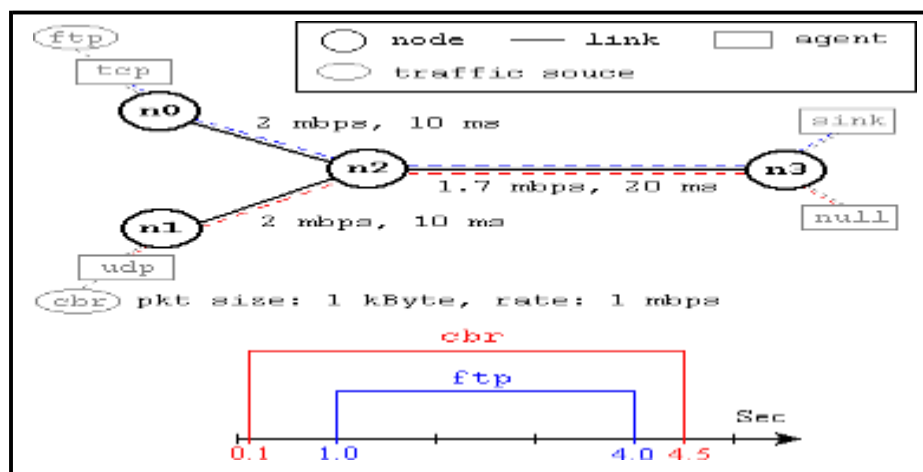
event	time	from node	to node	pkt type	pkt size	flags	fid	src addr	dst addr	seq num	pkt id
r : receive	(at to_node)										
+ : enqueue	(at queue)							src_addr	: node.port (3.0)		
- : dequeue	(at queue)							dst_addr	: node.port (0.0)		
d : drop	(at queue)										
r	1.3556	3	2	ack	40	-----	1	3.0	0.0	15	201
+	1.3556	2	0	ack	40	-----	1	3.0	0.0	15	201
-	1.3556	2	0	ack	40	-----	1	3.0	0.0	15	201
r	1.35576	0	2	tcp	1000	-----	1	0.0	3.0	29	199
+	1.35576	2	3	tcp	1000	-----	1	0.0	3.0	29	199
d	1.35576	2	3	tcp	1000	-----	1	0.0	3.0	29	199
+	1.356	1	2	cbr	1000	-----	2	1.0	3.1	157	207
-	1.356	1	2	cbr	1000	-----	2	1.0	3.1	157	207

Trace Format Example

Figure 4 . Details of Trace Window

1. The first field is the event type. It is given by one of four possible symbols r, +, -, d which correspond respectively to receive (at the output of the link), enqueued, dequeued and dropped.
2. The second field gives the time at which the event occurs.
3. Gives the input node of the link at which the event occurs.
4. Gives the output node of the link at which the event occurs.
5. Gives the packet type (eg CBR or TCP)
6. Gives the packet size
7. Some flags
8. This is the flow id (fid) of IPv6 that a user can set for each flow at the input OTcl script one can further use this field for analysis purposes; it is also used when specifying stream color for the NAMdisplay.
9. This is the source address given in the form of "node.port".
10. This is the destination address, given in the same form.
11. This is the network layer protocol's packet sequence number. Even though UDP implementations in a real network do not use sequence number, ns keeps track of UDP packet sequence number for analysis purposes
12. The last field shows the Unique id of the packet.

NS Simulation Script



This network consists of 4 nodes (n0, n1, n2, n3) as shown in Fig. 5. The duplex links between n0 and n2, and n1 and n2 have 2 Mbps of bandwidth and 10 ms of delay. The duplex link between n2 and n3 has 1.7 Mbps of bandwidth and 20 ms of delay. Each node uses a DropTail queue, of which the maximum size is 10. A "tcp" agent is attached to n0, and a connection is established to

a tcp "sink" agent attached to n3. As default, the maximum size of a packet that a "tcp" agent can generate is 1KByte.

A tcp "sink" agent generates and sends ACK packets to the sender (tcp agent) and frees the received packets. A "udp" agent that is attached to n1 is connected to a "null" agent attached to n3. A "null" agent just frees the packets received. A "ftp" and a "cbr" traffic generator are attached to "tcp" and "udp" agents respectively, and the "cbr" is configured to generate 1KByte packets at the rate of 1 Mbps.

The "cbr" is set to start at 0.1 sec and stop at 4.5 sec, and "ftp" is set to start at 1.0 sec and stop at 4.0 sec.

An Example Simulation Script #Create a simulator object

```
set ns [new Simulator]
```

```
#Define different colors for data flows (for NAM)
```

```
$ns color 1 Blue
```

```
$ns color 2 Red
```

```
#Open the NAM trace file
```

```
set nf [open out.nam w]
```

```
$ns namtrace-all
```

```
$nf
```

```
#Define a 'finish' procedure
```

```
proc finish
```

```
{}
```

```
{
```

```
    global
```

```
    ns nf
```

```
    $ns flush-trace
```

```
#Close the NAM trace file
```

```
close $nf
```

```
#Execute NAM on the trace file
```

```
exec nam out.nam & exit 0
```

```
}
```

```
#Create four nodes
```

```
set n0 [$ns node]
```

```
set n1 [$ns node]
```

```
set n2 [$ns node]
```

```
set n3 [$ns node]
```

```
#Create links between the nodes
```

```
$ns duplex-link $n0 $n2 2Mb 10ms DropTail
```

```
$ns duplex-link $n1 $n2 2Mb 10ms DropTail
```

```
$ns duplex-link $n2 $n3 1.7Mb 20ms DropTail
```

```
#Set Queue Size of link (n2-n3) to 10
```

```
$ns queue-limit $n2 $n3 10
```

```
#Give node position (for NAM)
```

```
$ns duplex-link-op $n0 $n2 orient right-down
```

```
$ns duplex-link-op $n1 $n2 orient right-up
```

```
$ns duplex-link-op $n2 $n3 orient right
#Monitor the queue for link (n2-n3). (for NAM)
$ns duplex-link-op $n2 $n3 queuePos 0.5
#Setup a TCP connection
set tcp [new Agent/TCP]
$tcp set class_ 2
$ns attach-agent $n0 $tcp
set sink [new Agent/TCPSink]
$ns attach-agent $n3 $sink
$ns connect $tcp $sink
$tcp set fid_ 1
#Setup a FTP over TCP connection
set ftp [new Application/FTP]
$ftp attach-agent $tcp
$ftp set type_ FTP
#Setup a UDP connection
set udp [newAgent/UDP]
$ns attach-agent $n1
$udp set null [newAgent/Null]
$ns attach-agent $n3 $null
$ns connect $udp $null
$udp set fid_ 2

#Setup a CBR over UDPconnection
set cbr [newApplication/Traffic/CBR]
$cbr attach-agent $udp
$$cbr set type_ CBR
$cbr set packet_size_1000
$cbr set rate_ 1mb
$cbr set random_ false
#Schedule events for the CBR and FTP agents
$ns at 0.1 "$cbr start"
$ns at 1.0 "$ftpstart"
$ns at 4.0 "$ftpstop"
$ns at 4.5 "$cbrstop"
$ns at 5.0 "finish"
#Print CBR packet size and interval
puts "CBR packet size = [$cbr set packet_size_]"
puts "CBR interval = [$cbr set interval_]"
#Run the simulation
$ns run
```

Program No. 1: Implement three nodes point – to – point network with duplex links between them. Set the queue size, vary the bandwidth and find the number of packets dropped.

Program Objective:

- Understand the Implementation of the Duplex link between the network.

Theory:

- Create a simulator object.
- We open a file for writing that is going to be used for the trace data.
- We now attach the agent to the nodes.
- Now we attach the application to run on top of the nodes
- We now connect the agent and the application for its working
- Set the simulation time
- The next step is to add a 'finish' procedure that closes the trace file and starts nam.

Source Code:

```
set ns [new Simulator]
set ntrace [open prog1.tr w]
$ns trace-all $ntrace
set namfile [open prog1.nam w]
$ns namtrace-all $namfile

proc countDrops {} {
    global ns ntrace namfile
    $ns flush-trace
    close $ntrace
    close $namfile
}

set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]

$ns duplex-link $n0 $n1 10Mb 10ms DropTail
$ns duplex-link $n1 $n2 5Mb 10ms DropTail

$ns queue-limit $n0 $n1 10
$ns queue-limit $n1 $n2 05

set tcp0 [new Agent/TCP]
$ns attach-agent $n0 $tcp0
```

```
set sink0 [new Agent/TCPSink]
$ns attach-agent $n2 $sink0

$ns connect $tcp0 $sink0

set cbr [new Application/Traffic/CBR]
$cbr set type_ CBR
$cbr set packetSize_ 100
$cbr set rate_ 1Mb
$cbr set random_ false
$cbr attach-agent $tcp0

$tcp0 set class_ 1

$ns at 1.0 "$cbr start"
$ns at 5.0 "Finish"

$ns run
```

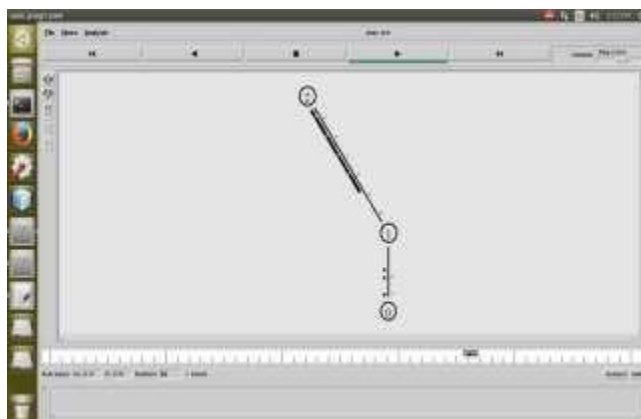
Output:

Steps for execution

1. Open gedit and type program. Program name should have the extension “.tcl”
student@cnp022:~/naveen\$ gedit prog1.tcl
2. Save the program.
3. Run the simulation program
student@cnp022:~/naveen\$ ns prog1.tcl
The number of packet drops is
5
4. Here “ns” indicates network simulator. We get the topology shown in the snapshot.
5. Now press the play button in the simulation window and the simulation will begin.
6. To see the trace file contents open the file as ,
student@cnp022:~/naveen\$ gedit prog1.tr

Topology

Snapshot 1:



Snapshot 2:

Trace File Snapshot

```

prog1.tr - gedit
1 0 1 tcp 40 ----- 1 0.0 2.0 0 0
1 0 1 tcp 40 ----- 1 0.0 2.0 0 0
r 1.010032 0 1 tcp 40 ----- 1 0.0 2.0 0 0
* 1.010032 1 2 tcp 40 ----- 1 0.0 2.0 0 0
- 1.010032 1 2 tcp 40 ----- 1 0.0 2.0 0 0
r 1.020096 1 2 tcp 40 ----- 1 0.0 2.0 0 0
* 1.020096 2 1 ack 40 ----- 1 2.0 0.0 0 1
- 1.020096 2 1 ack 40 ----- 1 2.0 0.0 0 1
r 1.03016 2 1 ack 40 ----- 1 2.0 0.0 0 1
- 1.03016 1 0 ack 40 ----- 1 2.0 0.0 0 1
r 1.03016 1 0 ack 48 ----- 1 2.0 0.0 0 1
- 1.040192 1 0 ack 48 ----- 1 2.0 0.0 0 1
* 1.040192 0 1 tcp 1040 ----- 1 0.0 2.0 1 2
- 1.040192 0 1 tcp 1040 ----- 1 0.0 2.0 1 2
* 1.040192 0 1 tcp 1040 ----- 1 0.0 2.0 2 3
- 1.040192 0 1 tcp 1040 ----- 1 0.0 2.0 2 3
r 1.051024 1 2 tcp 1040 ----- 1 0.0 2.0 1 2
* 1.051024 1 2 tcp 1040 ----- 1 0.0 2.0 1 2
- 1.051024 1 2 tcp 1040 ----- 1 0.0 2.0 1 2
r 1.051024 1 2 tcp 1040 ----- 1 0.0 2.0 1 2
* 1.051024 1 2 tcp 1040 ----- 1 0.0 2.0 2 3
- 1.051024 1 2 tcp 1040 ----- 1 0.0 2.0 2 3
r 1.062088 1 2 tcp 1040 ----- 1 0.0 2.0 1 2
* 1.062088 2 1 ack 40 ----- 1 2.0 0.0 1 4
- 1.062088 2 1 ack 40 ----- 1 2.0 0.0 1 4
r 1.064352 1 2 tcp 1040 ----- 1 0.0 2.0 2 3
* 1.064352 2 1 ack 40 ----- 1 2.0 0.0 2 5
- 1.064352 2 1 ack 40 ----- 1 2.0 0.0 2 5
r 1.072752 2 1 ack 40 ----- 1 2.0 0.0 1 4
* 1.072752 1 0 ack 40 ----- 1 2.0 0.0 1 4
- 1.072752 1 0 ack 40 ----- 1 2.0 0.0 1 4
r 1.074416 2 1 ack 40 ----- 1 2.0 0.0 2 5
* 1.074416 1 0 ack 40 ----- 1 2.0 0.0 2 5
- 1.074416 1 0 ack 40 ----- 1 2.0 0.0 2 5
r 1.082704 1 0 ack 40 ----- 1 2.0 0.0 1 4
* 1.082704 0 1 tcp 1040 ----- 1 0.0 2.0 3 6

```

Program No. 2: Implement transmission of ping messages/trace route over a network topology consisting of 6 nodes and find the number of packets dropped due to congestion

Program Objective:

- Understand the Implementation of the network topology consisting of n nodes.

Source Code:

```
set ns [new Simulator]
set ntrace [open prog2.tr w]
$ns trace-all $ntrace
set namfile [open prog2.nam w]
$ns namtrace-all $namfile
proc finish {} {
    global ns ntrace namfile
    $ns flush-trace
    close $ntrace
    close $namfile
    exec nam prog2.nam &
    puts "the number of ping packets dropped are"
    exec grep -c "^d" prog2.tr &
    exit 0
}
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]
set n6 [$ns node]
$ns duplex-link $n1 $n0 1Mb 120ms DropTail
$ns duplex-link $n2 $n0 1Mb 10ms DropTail
$ns duplex-link $n3 $n0 1Mb 10ms DropTail
$ns duplex-link $n4 $n0 1Mb 10ms DropTail
$ns duplex-link $n5 $n0 1Mb 10ms DropTail
$ns duplex-link $n6 $n0 1Mb 10ms DropTail
Agent/Ping instproc recv {from rtt} {
    set node_id [$self set node_id]
    puts "node $node_id received ping answer from $from round-trip-time $rtt ms"
}
set p1 [new Agent/Ping]
set p2 [new Agent/Ping]
set p3 [new Agent/Ping]
set p4 [new Agent/Ping]
set p5 [new Agent/Ping]
```

```
set p6 [new Agent/Ping]
$p1 set node_id 1
$p2 set node_id 2
$p3 set node_id 3
$p4 set node_id 4
$p5 set node_id 5
$p6 set node_id 6
$nns attach-agent $n1 $p1
$nns attach-agent $n2 $p2
$nns attach-agent $n3 $p3
$nns attach-agent $n4 $p4
$nns attach-agent $n5 $p5
$nns attach-agent $n6 $p6
$nns queue-limit $n0 $n4 3
$nns queue-limit $n0 $n5 1
$nns queue-limit $n0 $n6 1
$nns connect $p1 $p4
$nns connect $p2 $p5
$nns connect $p3 $p6
$nns at 0.1 "$p1 send"
$nns at 0.3 "$p2 send"
$nns at 0.5 "$p3 send"
$nns at 1.0 "$p4 send"
$nns at 1.2 "$p5 send"
$nns at 1.4 "$p6 send"
$nns at 2.0 "finish"

$nns run
```

Output:

Steps for execution

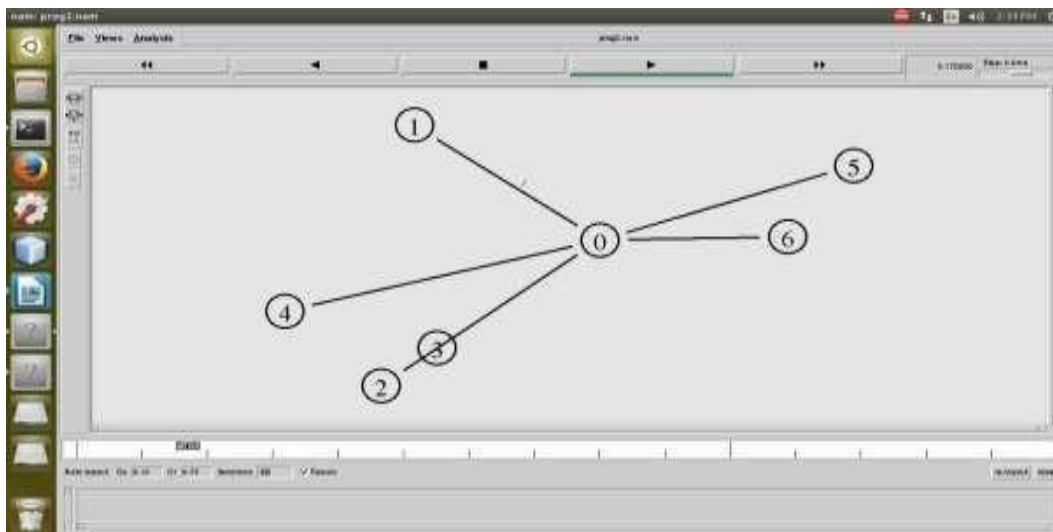
1. Open gedit and type program. Program name should have the extension “.tcl”
student@cnp022:~/naveen\$ gedit prog2.tcl
2. Save the program.
3. Run the simulation program
student@cnp022:~/naveen\$ ns prog2.tcl

**node 1 received ping answer from 4 round-trip-time
262.0 ms node 4 received ping answer from 1 round-
trip-time 262.0 ms the number of ping packets dropped
are 4**

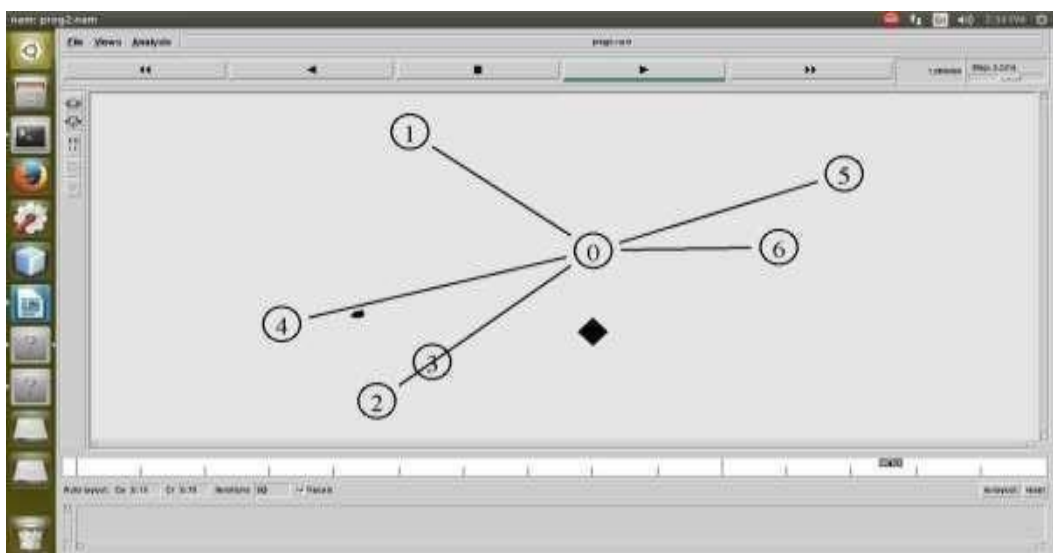
4. Here “ns” indicates network simulator. We get the topology shown in the snapshot.
5. Now press the play button in the simulation window and the simulation will begin.
6. To see the trace file contents open the file as ,

Topology

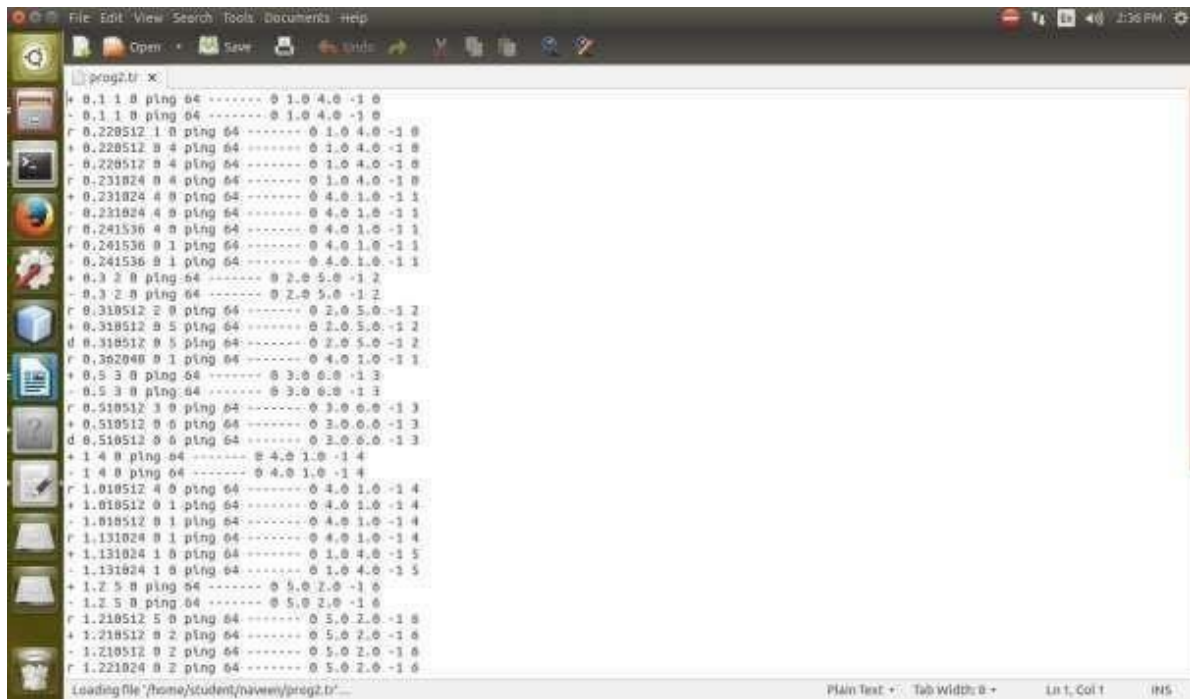
Snapshot 1:



Snapshot 2:



Snapshot 3:Trace File snapshot



```
File Edit View Search Tools Documents Help
prog2.tr x
+ 0.1 1 0 ping 64 ----- 0 1.0 4.0 -1 0
- 0.1 1 0 ping 64 ----- 0 1.0 4.0 -1 0
r 0.220512 1 0 ping 64 ----- 0 1.0 4.0 -1 0
+ 0.220512 0 4 ping 64 ----- 0 1.0 4.0 -1 0
- 0.220512 0 4 ping 64 ----- 0 1.0 4.0 -1 0
r 0.231024 0 4 ping 64 ----- 0 1.0 4.0 -1 0
+ 0.231024 0 0 ping 64 ----- 0 4.0 1.0 -1 1
- 0.231024 0 0 ping 64 ----- 0 4.0 1.0 -1 1
r 0.241536 0 0 ping 64 ----- 0 4.0 1.0 -1 1
+ 0.241536 0 1 ping 64 ----- 0 4.0 1.0 -1 1
- 0.241536 0 1 ping 64 ----- 0 4.0 1.0 -1 1
+ 0.3 2 0 ping 64 ----- 0 2.0 5.0 -1 2
- 0.3 2 0 ping 64 ----- 0 2.0 5.0 -1 2
r 0.310512 2 0 ping 64 ----- 0 2.0 5.0 -1 2
+ 0.310512 0 5 ping 64 ----- 0 2.0 5.0 -1 2
d 0.310512 0 5 ping 64 ----- 0 2.0 5.0 -1 2
r 0.302040 0 1 ping 64 ----- 0 4.0 1.0 -1 1
+ 0.5 3 0 ping 64 ----- 0 3.0 0.0 -1 3
- 0.5 3 0 ping 64 ----- 0 3.0 0.0 -1 3
r 0.510512 3 0 ping 64 ----- 0 3.0 0.0 -1 3
+ 0.510512 0 0 ping 64 ----- 0 3.0 0.0 -1 3
d 0.510512 0 0 ping 64 ----- 0 3.0 0.0 -1 3
+ 1 4 0 ping 64 ----- 0 4.0 1.0 -1 4
- 1 4 0 ping 64 ----- 0 4.0 1.0 -1 4
r 1.010512 4 0 ping 64 ----- 0 4.0 1.0 -1 4
+ 1.010512 0 1 ping 64 ----- 0 4.0 1.0 -1 4
- 1.010512 0 1 ping 64 ----- 0 4.0 1.0 -1 4
r 1.131024 0 1 ping 64 ----- 0 4.0 1.0 -1 4
+ 1.131024 1 0 ping 64 ----- 0 1.0 4.0 -1 5
- 1.131024 1 0 ping 64 ----- 0 1.0 4.0 -1 5
+ 1.2 5 0 ping 64 ----- 0 5.0 2.0 -1 6
- 1.2 5 0 ping 64 ----- 0 5.0 2.0 -1 6
r 1.210512 5 0 ping 64 ----- 0 5.0 2.0 -1 6
+ 1.210512 0 2 ping 64 ----- 0 5.0 2.0 -1 6
- 1.210512 0 2 ping 64 ----- 0 5.0 2.0 -1 6
r 1.221024 0 2 ping 64 ----- 0 5.0 2.0 -1 6
Loading file "/home/student/naveen/prog2.tr"...
```

Program No. 3: Implement an Ethernet LAN using n nodes and set multiple traffic nodes and plot congestion window for different source / destination.**Program Objective:**

- Understand the Implementation of the Ethernet LAN using n nodes.

Source Code:

```
set ns [new Simulator]
set mytrace [open prog3.tr w]
$ns trace-all $mytrace
set myNAM [open prog3.nam w]
$ns namtrace-all $myNAM

set n0 [$ns node]
$n0 color "RED"
$n0 label "src1"
set n1 [$ns node]
set n2 [$ns node]
$n2 color "BLUE"
$n2 label "src2"
set n3 [$ns node]
$n3 color "YELLOW"
$n3 label "dest2"
set n4 [$ns node]
set n5 [$ns node]
$n5 color "MAGENTA"
$n5 label "dest1"

$ns make-lan "$n0 $n1 $n2 $n3 $n4" 100Mb 100ms LL Queue/DropTail Mac/802_3
$ns duplex-link $n4 $n5 1Mb 1ms DropTail

set tcp0 [new Agent/TCP]
$ns attach-agent $n0 $tcp0
set ftp0 [new Application/FTP]
$ftp0 attach-agent $tcp0
$ftp0 set packetSize_ 500
$ftp0 set interval_ 0.0001
set sink5 [new Agent/TCPSink]
$ns attach-agent $n5 $sink5
$ns connect $tcp0 $sink5

set tcp2 [new Agent/TCP]
$ns attach-agent $n2 $tcp2
```

```
$ftp2 attach-agent $tcp2
$ftp2 set packetSize_ 600
$ftp2 set interval_ 0.001
set sink3 [new Agent/TCPSink]
$ns attach-agent $n3 $sink3
$ns connect $tcp2 $sink3
```

```
set file1 [open file1.tr w]
$tcp0 attach $file1
set file2 [open file2.tr w]
$tcp2 attach $file2
# Trace congestion window
$tcp0 trace cwnd_
$tcp2 trace cwnd_
```

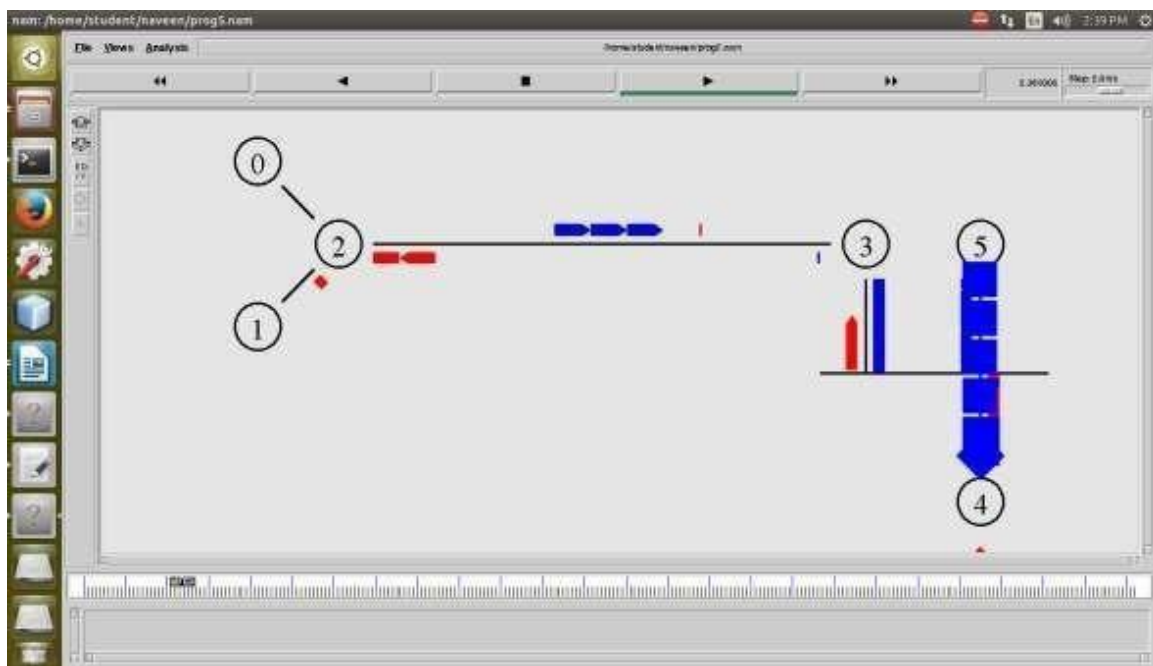
```
proc finish {} {
    global ns mytrace MyNAM
    $ns flush-trace
    close $mytrace
    close $myNAM
    exec nam prog3.nam &
}
```

```
$ns at 0.1 "$ftp0 start"
$ns at 5.0 "$ftp0 stop"
$ns at 7.0 "$ftp0 start"
$ns at 14.0 "$ftp0 stop"
$ns at 0.2 "$ftp2 start"
$ns at 8.2 "$ftp2 stop"
$ns at 10.0 "$ftp2 start"
$ns at 15.0 "$ftp2 stop"
```

```
$ns run
```

Output:**Steps for execution**

1. Open gedit and type program. Program name should have the extension “.tcl”
student@cnp022:~/naveen\$ gedit prog3.tcl
2. Save the program.
3. Run the simulation program
student@cnp022:~/naveen\$ ns prog3.tcl
4. Here “ns” indicates network simulator. We get the topology shown in the snapshot.
5. Now press the play button in the simulation window and the simulation will begin.
6. To see the trace file contents open the file as ,
student@cnp022:~/naveen\$ gedit prog3.tr

Topology**Snapshot 1:**

Program 4. Write a program for error detecting code using CRC-CCITT (16-bits).

Program Objective:

- Understand the operation of CRC-CCITT.

Whenever digital data is stored or interfaced, data corruption might occur. Since the beginning of computer science, developers have been thinking of ways to deal with this type of problem. For serial data they came up with the solution to attach a parity bit to each sent byte. This simple detection mechanism works if an odd number of bits in a byte changes, but an even number of false bits in one byte will not be detected by the parity check.

To overcome this problem developer have searched for mathematical sound mechanisms to detect multiple false bits. The **CRC** calculation or *cyclic redundancy check* was the result of this. Nowadays CRC calculations are used in all types of communications. All packets sent over a network connection are checked with a CRC. Also each data block on your hard disk has a CRC value attached to it.

Modern computer world cannot do without these CRC calculations. So let's see why they are so widely used. The answer is simple; they are powerful, detect many types of errors and are extremely fast to calculate especially when dedicated hardware chips are used.

The idea behind CRC calculation is to look at the data as one large binary number. This number is divided by a certain value and the remainder of the calculation is called the CRC. Dividing in the CRC calculation at first looks to cost a lot of computing power, but it can be performed very quickly if we use a method similar to the one learned at school. We will as an example calculate the remainder for the character 'm'—which is 1101101 in binary notation— by dividing it by 19 or 10011. Please note that 19 is an odd number.

This is necessary as we will see further on. Please refer to your schoolbooks as the binary calculation method here is not very different from the decimal method you learned when you were young. It might only look a little bit strange. Also notations differ between countries, but the method is similar.

$$\begin{array}{r}
 101 = 5 \\
 \text{-----} \\
 10011 \, / \, 1101101 \\
 10011 \, | \, | \\
 \text{-----} \, | \, | \\
 10000 \, | \\
 00000 \, | \\
 \text{-----} \, | \\
 10000 \, 1 \\
 10011 \\
 \text{-----} \\
 1110 = 14 = \text{remainder}
 \end{array}$$

With decimal calculations you can quickly check that 109 divided by 19 gives a quotient of 5 with 14

as the remainder. But what we also see in the scheme is that every bit extra to check only costs one binary comparison and in 50% of the cases one binary subtraction. You can easily increase the number of bits of the test data string—for example to 56 bits if we use our example value "Lammert"—and the result can be calculated with 56 binary comparisons and an average of 28 binary subtractions. This can be implemented in hardware directly with only very few transistors involved. Also software algorithms can be very efficient.

All of the CRC formulas you will encounter are simply checksum algorithms based on modulo-2 binary division where we ignore carry bits and in effect the subtraction will be equal to an *exclusive or* operation. Though some differences exist in the specifics across different CRC formulas, the basic mathematical process is always the same:

- The message bits are appended with c zero bits; this *augmented message* is the dividend
- A predetermined $c+1$ -bit binary sequence, called the *generator polynomial*, is the divisor. The checksum is the c -bit remainder that results from the division operation

Table 1 lists some of the most commonly used generator polynomials for 16- and 32-bit CRCs. Remember that the width of the divisor is always one bit wider than the remainder. So, for example, you'd use a 17-bit generator polynomial whenever a 16-bit checksum is required.

	CRC-CCITT	CRC-16	CRC-32
Checksum			
Width	16 bits	16 bits	32 bits
Generator Polynomial	1000100000010000 1	11000000000000101	100000100110000010001110110110111

International Standard

CRC Polynomials

Algorithm: -

1. Given a bit string, append 0^s to the end of it (the number of 0^s is the same as the degree of the generator polynomial) let $B(x)$ be the polynomial corresponding to B .
2. Divide $B(x)$ by some agreed on polynomial $G(x)$ (generator polynomial) and determine the remainder $R(x)$. This division is to be done using Modulo 2 Division.
3. Define $T(x) = B(x) - R(x)$ ($T(x)/G(x) \Rightarrow$ remainder 0)
4. Transmit T , the bit string corresponding to $T(x)$.
Let T' represent the bit stream the receiver gets and $T'(x)$ the associated polynomial. The receiver divides $T'(x)$ by $G(x)$. If there is a 0 remainder, the receiver concludes $T = T'$ and no error occurred otherwise, the receiver concludes an error occurred and requires a retransmission

Source Code:

```
#include <stdio.h>
#include <string.h>
#define MAX_DATA 100
#define MAX_GEN 20
char data[MAX_DATA];
char check_value[MAX_GEN];
char gen_poly[MAX_GEN];
int data_length;
int i, j;
#define N strlen(gen_poly)
/* XOR operation */
void XOR()
{
    for (j = 1; j < N; j++)
    {
        check_value[j] =
            ((check_value[j] == gen_poly[j]) ? '0' : '1');
    }
}
/* CRC calculation */
void crc()
{
    for (i = 0; i < N; i++)
        check_value[i] = data[i];
    do
    {
        if (check_value[0] == '1')
            XOR();
        for (j = 0; j < N - 1; j++)
            check_value[j] = check_value[j + 1];
        check_value[j] = data[i++];
    } while (i <= data_length + N - 1);
}
/* Receiver */
void receiver()
{
    printf("\nEnter the received data: ");
    scanf("%s", data);
    printf("\n-----");
    printf("\nData received: %s", data);
    crc();
    for (i = 0; (i < N - 1) && (check_value[i] != '1'); i++);
    if (i < N - 1)
        printf("\nError detected\n");
    else
        printf("\nNo error detected\n");
}
int main()
{
```

```
printf("\nEnter data to be transmitted: ");
scanf("%s", data);
printf("\nEnter the Generating polynomial: ");
scanf("%s", gen_poly);
data_length = strlen(data);
/* Append N-1 zeros */
for (i = data_length; i < data_length + N - 1; i++)
    data[i] = '0';
data[data_length + N - 1] = '\0';
printf("\n-----");
printf("\nData padded with n-1 zeros: %s", data);
printf("\n-----");
/* Calculate CRC */
crc();
check_value[N - 1] = '\0';
printf("\nCRC or Check value is: %s", check_value);
/* Append CRC to original data */
for (i = data_length; i < data_length + N - 1; i++)
    data[i] = check_value[i - data_length];
data[data_length + N - 1] = '\0';
printf("\n-----");
printf("\nFinal data to be sent: %s", data);
printf("\n-----");
/* Receiver */
receiver();
return 0;
```

Output :

aiml@aiml-16:~\$ gedit pg4.c

aiml@aiml-16:~\$ gcc pg4.c

aiml@aiml-16:~\$./pg4

Enter data to be transmitted: 1001101

Enter the Generating polynomial: 1001

Data padded with n-1 zeros: 1001101000

CRC or Check value is: 101

Final data to be sent: 1001101101

Enter the received data: 1001101101

Data received: 1001101101

No error detected

aiml@aiml-16:~\$./pg4

Enter data to be transmitted: 100110011000

Enter the Generating polynomial: 1011

Data padded with n-1 zeros: 100110011000000

CRC or Check value is: 000

Final data to be sent: 100110011000000

Enter the received data: 100110011001

Data received: 100110011001

Error detected

Program 5. Develop a simple data link layer that performs the flow control using the sliding window protocol, and loss recovery using the Go-Back-N mechanism.**Source Code:**

```
#include <stdio.h>
int main()
{
    int w, i, f;
    int frames[50];
    printf("Enter window size: ");
    scanf("%d", &w);
    printf("\nEnter number of frames to transmit: ");
    scanf("%d", &f);
    printf("\nEnter %d frames: ", f);
    for (i = 1; i <= f; i++)
    {
        scanf("%d", &frames[i]);
    }
    printf("\nWith sliding window protocol, the frames will be sent "
        "in the following manner (assuming no corruption of frames):\n\n");
    printf("After sending %d frames at each stage, sender waits for "
        "acknowledgement sent by the receiver.\n\n", w);
    for (i = 1; i <= f; i++)
    {
        printf("%d ", frames[i]);
        if (i % w == 0)
        {
            printf("\n");
            printf("Acknowledgement of above frames sent is received "
                "by sender.\n\n");
        }
    }
    if (f % w != 0)
    {
        printf("\nAcknowledgement of above frames sent is received "
            "by sender.\n");
    }
    return 0;
}
```

Output :

```
aiml@aiml-16:~$ gedit pg5.c
aiml@aiml-16:~$ gcc pg5.c
aiml@aiml-16:~$ ./pg5
Enter window size: 3
```

Enter number of frames to transmit: 8

Enter 8 frames: 1

2
3
4
5
6
7
8

With sliding window protocol, the frames will be sent in the following manner (assuming no corruption of frames):

After sending 3 frames at each stage, sender waits for acknowledgement sent by the receiver.

1 2 3

Acknowledgement of above frames sent is received by sender.

4 5 6

Acknowledgement of above frames sent is received by sender.

7 8

Acknowledgement of above frames sent is received by sender.

Program 6: Write a program to find the shortest path between vertices using bellman-ford algorithm.

Program Objective:

- Understand the Implementation of the shortest path for bellman-ford algorithm.

Distance Vector Algorithm is a decentralized routing algorithm that requires that each router simply inform its neighbors of its routing table. For each network path, the receiving routers pick the neighbor advertising the lowest cost, then add this entry into its routing table for re-advertisement. To find the shortest path, Distance Vector Algorithm is based on one of two basic algorithms: the Bellman-Ford and the Dijkstra algorithms.

Routers that use this algorithm have to maintain the distance tables (which is a one-dimension array - "a vector"), which tell the distances and shortest path to sending packets to each node in the network. The information in the distance table is always up date by exchanging information with the neighboring nodes. The number of data in the table equals to that of all nodes in networks (excluded itself).

The columns of table represent the directly attached neighbors whereas the rows represent all destinations in the network. Each data contains the path for sending packets to each destination in the network and distance/or time to transmit on that path (we call this as "cost"). The measurements in this algorithm are the number of hops, latency, the number of outgoing packets, etc.

The Bellman-Ford algorithm is an algorithm that computes shortest paths from a single source vertex to all of the other vertices in a weighted digraph. It is slower than Dijkstra's algorithm for the same problem, but more versatile, as it is capable of handling graphs in which some of the edge weights are negative numbers. Negative edge weights are found in various applications of graphs, hence the usefulness of this algorithm.

If a graph contains a "negative cycle" (i.e. a cycle whose edges sum to a negative value) that is reachable from the source, then there is no cheapest path: any path that has a point on the negative cycle can be made cheaper by one more walk around the negative cycle. In such a case, the Bellman-Ford algorithm can detect negative cycles and report their existence

Implementation Algorithm:

1. send my routing table to all my neighbours whenever my link table changes
2. when I get a routing table from a neighbour on port P with link metric M:
 - a. add L to each of the neighbor's metrics
 - b. for each entry (D, P', M') in the updated neighbor's table:
 - i. if I do not have an entry for D, add (D, P, M') to my routing table
 - ii. if I have an entry for D with metric M", add (D, P, M') to my routing table if $M' < M''$
3. if my routing table has changed, send all the new entries to all my neighbor

Source Code:

```
import java.util.Scanner;
public class BellmanFord{
    private int[] D;
    private int n;
    public static final int MAX_VALUE = 999;
    public BellmanFord(int n) {
        this.n = n;
        D = new int[n + 1];
    }
    public void shortest(int s, int[][] A) {
        // Initialize distances
        for (int i = 1; i <= n; i++) {
            D[i] = MAX_VALUE;
        }
        D[s] = 0;
        // Relax edges repeatedly
        for (int k = 1; k <= n - 1; k++) {
            for (int i = 1; i <= n; i++) {
                for (int j = 1; j <= n; j++) {
                    if (A[i][j] != MAX_VALUE) {
                        if (D[i] + A[i][j] < D[j]) {
                            D[j] = D[i] + A[i][j];
                        }
                    }
                }
            }
        }
        // Check for negative weight cycles
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                if (A[i][j] != MAX_VALUE) {
                    if (D[i] + A[i][j] < D[j]) {
                        System.out.println("The Graph contains a negative edge cycle");
                        return;
                    }
                }
            }
        }
        // Print shortest distances
        for (int i = 1; i <= n; i++) {
            System.out.println("Distance of source " + s + " to " + i + " is " + D[i]);
        }
    }
    public static void main(String[] args) {
        int n = 0, s;
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number of vertices");
        n = sc.nextInt();
        int[][] A = new int[n + 1][n + 1];
    }
}
```

```
System.out.println("Enter the Weighted matrix");
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        A[i][j] = sc.nextInt();
        if (i == j) {
            A[i][j] = 0;
        } else if (A[i][j] == 0 && i != j) {
            A[i][j] = MAX_VALUE;
        }
    }
}
System.out.println("Enter the source vertex");
s = sc.nextInt();

BellmanFord b = new BellmanFord(n);
b.shortest(s, A);
sc.close();
}
```

Output:

```
aiml@aiml-16:~$ gedit BellmanFord.java
aiml@aiml-16:~$ javac BellmanFord.java
aiml@aiml-16:~$ java BellmanFord
Enter the number of vertices
4
Enter the Weighted matrix
0 5 0 0
5 0 3 4
0 3 0 2
0 4 2 0
Enter the source vertex
2
Distance of source 2 to 1 is 5
Distance of source 2 to 2 is 0
Distance of source 2 to 3 is 3
Distance of source 2 to 4 is 4
```

Program 7: Using TCP/IP sockets, write a client – server program to make the client send the file name and to make the server send back the contents of the requested file if present.

Program Objective:

- **Understand the Implementation of the transport layer protocols.**

Socket is an interface which enables the client and the server to communicate and pass on information from one another. Sockets provide the communication mechanism between two computers using TCP. A client program creates a socket on its end of the communication and attempts to connect that socket to a server. When the connection is made, the server creates a socket object on its end of the communication. The client and the server can now communicate by writing to and reading from the socket.

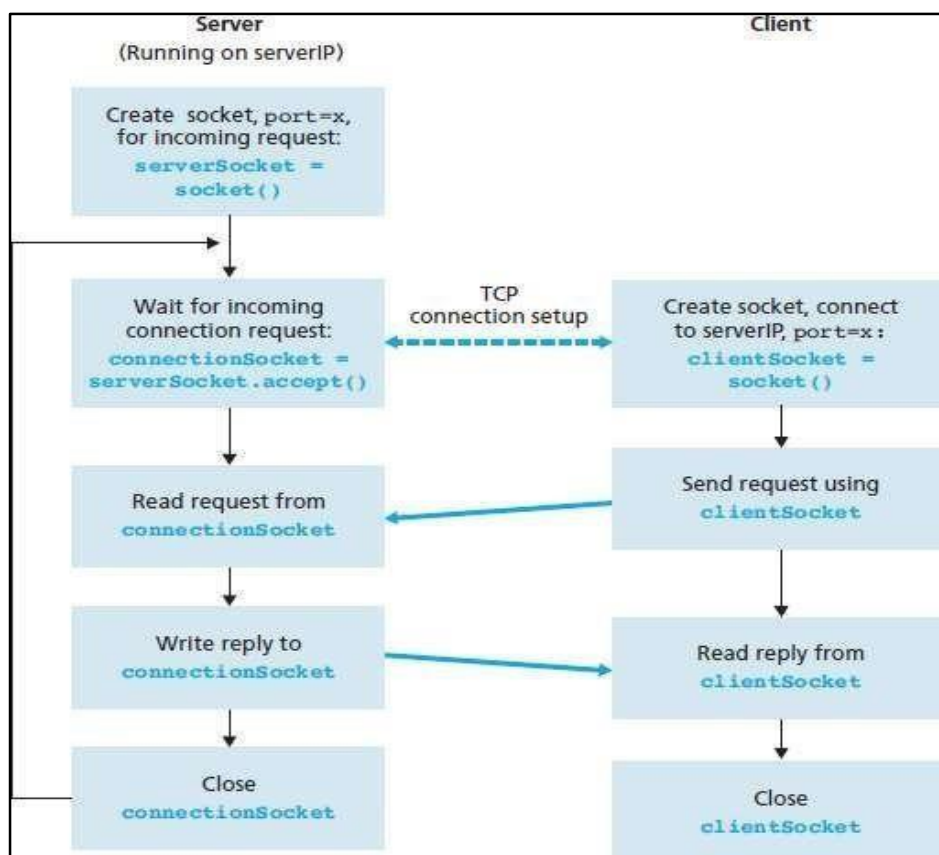


Fig. 6 The client-server application using TCP

Algorithm (Client Side)

1. Start.
2. Create a socket using socket() system call.
3. Connect the socket to the address of the server using connect() system call.
4. Send the filename of required file using send() system call.

5. Read the contents of the file sent by server by recv() system call.
6. Stop.

Algorithm (Server Side)

1. Start.
2. Create a socket using socket() system call.
3. Bind the socket to an address using bind() system call.
4. Listen to the connection using listen() system call.
5. accept connection using accept()
6. Receive filename and transfer contents of file with client.
7. Stop.

TCP Client side:

```
import java.net.*;
import java.io.*;
public class TCPClient
{
    public static void main(String args[])
    {
        try
        {
            Socket sock = new Socket("127.0.0.1", 4000);
            System.out.println("Connected to server");
            BufferedReader keyRead =
                new BufferedReader(new InputStreamReader(System.in));
            System.out.print("Enter the filename: ");
            String fname = keyRead.readLine();
            PrintWriter pwrite =
                new PrintWriter(sock.getOutputStream(), true);
            // Send filename to server
            pwrite.println(fname);
            BufferedReader socketRead =
                new BufferedReader(
                    new InputStreamReader(sock.getInputStream()));
            System.out.println("\nFile contents:");
            String str;
            while ((str = socketRead.readLine()) != null)
            {
                System.out.println(str);
            }
            socketRead.close();
            pwrite.close();
            keyRead.close();
            sock.close();
        }
        catch (ConnectException e)
        {
            System.out.println("Error: Server is not running.");
            System.out.println("Start TCPServer first, then run TCPClient.");
        }
    }
}
```

```
        catch (Exception e)
        {
            System.out.println("Error: " + e);
        }
    }
}
```

TCP Server side:

```
import java.net.*;
import java.io.*;
public class TCPServer
{
    public static void main(String args[]) throws Exception
    {
        ServerSocket sersock = new ServerSocket(4000);
        System.out.println("Server ready for Connection");
        Socket sock = sersock.accept();
        System.out.println("Connection is Successful and waiting for file");
        InputStream istream = sock.getInputStream();
        BufferedReader fileRead =
            new BufferedReader(new InputStreamReader(istream));
        String fname = fileRead.readLine();
        OutputStream ostream = sock.getOutputStream();
        PrintWriter pwrite = new PrintWriter(ostream, true);
        File file = new File(fname);
        if (file.exists())
        {
            BufferedReader contentRead =
                new BufferedReader(new FileReader(file));
            String str;
            while ((str = contentRead.readLine()) != null)
            {
                pwrite.println(str);
            }
        }
    }
}
```

```
    }  
    contentRead.close();  
}  
else  
{  
    pwrite.println("File not found: " + fname);  
}  
sock.close();  
sersock.close();  
pwrite.close();  
fileRead.close();  
}  
}
```

Note: Create two different files TcpClient.java and TcpServer.java. Follow the steps given:

1. Open a terminal run the server program and provide the filename to send
2. Open one more terminal run the client program and provide the IP address of the server. We can give local host address “127.0.0.1” as it is running on same machine or give the IP address of the machine.
3. Send any start bit to start sending file.

First Method of Executing TCP/IP socket based program**Output:****TCP Server side:**

```
aiml@aiml-16:~$ gedit TCPServer.java
aiml@aiml-16:~$ gedit TCPClient.java
aiml@aiml-16:~$ javac TCPServer.java
aiml@aiml-16:~$ javac TCPClient.java
aiml@aiml-16:~$ java TCPServer
Server ready for Connection
Connection is Successful and waiting for chatting
```

TCPClient

Next run TCPClient program

Enter the filename

```
/home/student/naveen/abc.txt
hello atme college of engineering
```

Second Method of Executing TCP/IP socket based program**Output:****TCP Client side:**

```
aiml@aiml-16:~$ java TCPClient
Enter the filename
abc.txt
hello wellcome to atme college of
engineering
aiml@aiml-16:~$
```

Program 8: Write a program on datagram socket for client/server to display the messages on client side, typed at the server side.

Program Objective:

- Understand the Implementation of the data link layer protocols.
- In computer networking, the UDP stands for User Datagram Protocol. The David P. Reed developed the UDP protocol in 1980. It is defined in RFC 768, and it is a part of the TCP/IP protocol, so it is a standard protocol over the internet. The UDP protocol allows the computer applications to send the messages in the form of datagrams from one machine to another machine over the Internet Protocol (IP) network.
- The UDP is an alternative communication protocol to the TCP protocol (transmission control protocol). Like TCP, UDP provides a set of rules that governs how the data should be exchanged over the internet. The UDP works by encapsulating the data into the packet and providing its own header information to the packet. Then, this UDP packet is encapsulated to the IP packet and sent off to its destination. Both the TCP and UDP protocols send the data over the internet protocol network, so it is also known as TCP/IP and UDP/IP.
- There are many differences between these two protocols. UDP enables the process to process communication, whereas the TCP provides host to host communication. Since UDP sends the messages in the form of datagrams, it is considered the best-effort mode of communication.
- TCP sends the individual packets, so it is a reliable transport medium. Another difference is that the TCP is a connection-oriented protocol whereas, the UDP is a connectionless protocol as it does not require any virtual circuit to transfer the data.
- UDP also provides a different port number to distinguish different user requests and also provides the checksum capability to verify whether the complete data has arrived or not; the IP layer does not provide these two services.

Java supports datagram communication through the following classes:

- DatagramPacket
- DatagramSocket

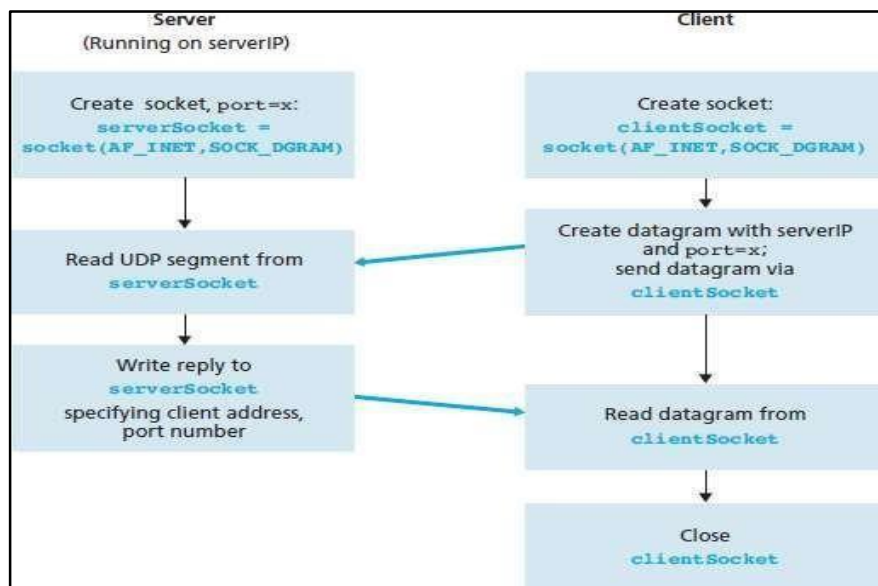


Fig -7-UDP client/server communication flow:

/* Datagram Socket Program */
UDP Server

```

import java.io.*;
import java.net.*;
public class UDPServer {
    public static void main(String[] args) {
        DatagramSocket skt = null;
        try {
            System.out.println("server is started");
            skt = new DatagramSocket(6788);
            byte[] buffer = new byte[1000];

            while (true) {
                DatagramPacket request = new DatagramPacket(buffer, buffer.length);
                skt.receive(request);
                String[] message = (new String(request.getData())).split(" ");
                byte[] sendMsg = (message[1].toUpperCase() + " from server to client").getBytes();

                DatagramPacket reply = new DatagramPacket(sendMsg, sendMsg.length,
                request.getAddress(), request.getPort());
                skt.send(reply);
            }
        } catch (Exception ex) {

            System.out.println(ex.getMessage());

        } finally {
            if (skt != null && !skt.isClosed()) {
                skt.close();
            }
        }
    }
}
  
```

```
    }  
  }  
}
```

UDP Client

```
import java.io.*;  
import java.net.*;  
public class UDPClient {  
    public static void main(String[] args) {  
        DatagramSocket skt = null;  
        try {  
            skt = new DatagramSocket();  
            // Send message to server  
            String msg = "atme college";  
            byte[] b = msg.getBytes();  
            InetAddress host = InetAddress.getByName("127.0.0.1");  
            int serverPort = 6788;  
  
            DatagramPacket request = new DatagramPacket(b, b.length, host, serverPort);  
            skt.send(request);  
  
            // Prepare buffer to receive response  
            byte[] buffer = new byte[1024];  
            DatagramPacket reply = new DatagramPacket(buffer, buffer.length);  
            skt.receive(reply);  
  
            // Print the received message  
            System.out.println("client received: " + new String(reply.getData(), 0, reply.getLength()));  
        } catch (Exception ex) {  
            ex.printStackTrace();  
        } finally {  
            if (skt != null && !skt.isClosed()) {  
                skt.close();  
            }  
        }  
    }  
}
```

Method of Executing datagram socket program for client/server to display the messages on client side, typed at the server side.

Output:

UDPServer

```
aiml@aiml-16:~$ gedit UDPServer.java
aiml@aiml-16:~$ gedit UDPClient.java
aiml@aiml-16:~$ javac UDPServer.java
aiml@aiml-16:~$ javac UDPClient.java
aiml@aiml-16:~$ java UDPServer
server is started
```

UDPClient

```
aiml@aiml-16:~$ java UDPClient
client received: COLLEGE from server to client
aiml@aiml-16:~$
```

Program 9: Write a program for simple RSA algorithm to encrypt and decrypt the data.**Program Objective:**

- Understand the operation of RSA algorithm.

The RSA algorithm can be used for both public key encryption and digital signatures. Its security is based on the difficulty of factoring large integers.

The RSA algorithm's efficiency requires a fast method for performing the modular exponentiation operation. A less efficient, conventional method includes raising a number (the input) to a power (the secret or public key of the algorithm, denoted e and d , respectively) and taking the remainder of the division with N . A straight-forward implementation performs these two steps of the operation sequentially: first, raise it to the power and second, apply modulo.

A very simple example of RSA encryption

This is an extremely simple example using numbers you can work out on a pocket calculator (those of you over the age of 35 can probably even do it by hand on paper).

1. Select primes $p = 11$, $q = 3$.
2. $n = pq = 11 \cdot 3 = 33$ $\phi = (p-1)(q-1) = 10 \cdot 2 = 20$
3. Choose $e=3$ Check $\gcd(e, p-1) = \gcd(3, 10) = 1$ (i.e. 3 and 10 have no common factors except 1), and check $\gcd(e, q-1) = \gcd(3, 2) = 1$
therefore $\gcd(e, \phi) = \gcd(e, (p-1)(q-1)) = \gcd(3, 20) = 1$
4. Compute d such that $ed \equiv 1 \pmod{\phi}$
i.e. compute $d = e^{-1} \pmod{\phi} = 3^{-1} \pmod{20}$
i.e. find a value for d such that ϕ divides $(ed-1)$
i.e. find d such that 20 divides $3d-1$. Simple testing ($d = 1, 2, \dots$) gives $d = 7$
Check: $ed-1 = 3 \cdot 7 - 1 = 20$, which is divisible by ϕ .
5. Public key = $(n, e) = (33, 3)$
Private key = $(n, d) = (33, 7)$.

This is actually the smallest possible value for the modulus n for which the RSA algorithm works.

Now say we want to encrypt the message $m = 7$,

$$c = m^e \pmod{n} = 7^3 \pmod{33} = 343 \pmod{33} = 13.$$

Hence the ciphertext $c = 13$.

To check decryption we compute

$$m' = c^d \bmod n = 13^7 \bmod 33 = 7.$$

Note that we don't have to calculate the full value of 13 to the power 7 here. We can make use of the fact that $a = bc \bmod n = (b \bmod n)(c \bmod n) \bmod n$ so we can break down a potentially large number into its components and combine the results of easier, smaller calculations to calculate the final value.

One way of calculating m' is as follows:-

$$\begin{aligned} m' &= 13^7 \bmod 33 = 13^{(3+3+1)} \bmod 33 = 13^3 \cdot 13^3 \cdot 13 \bmod 33 \\ &= (13^3 \bmod 33) \cdot (13^3 \bmod 33) \cdot (13 \bmod 33) \bmod 33 \\ &= (2197 \bmod 33) \cdot (2197 \bmod 33) \cdot (13 \bmod 33) \bmod 33 = 19 \cdot 19 \cdot 13 \bmod 33 \\ &= 4693 \bmod 33 \\ &= 7. \end{aligned}$$

Now if we calculate the cipher text c for all the possible values of m (0 to 32), we get

m	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
c	0	1	8	27	31	26	18	13	17	3	10	11	12	19	5	9	4

m	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
c	29	24	28	14	21	22	23	30	16	20	15	7	2	6	25	32

Note that all 33 values of m (0 to 32) map to a unique code c in the same range in a sort of random manner. In this case we have nine values of m that map to the same value of c - these are known as unconcealed messages. $m = 0$ and 1 will always do this for any N , no matter how large. But in practice, higher values shouldn't be a problem when we use large values for N .

If we wanted to use this system to keep secrets, we could let $A=2$, $B=3$, ..., $Z=27$. (We specifically avoid 0 and 1 here for the reason given above). Thus the plaintext message "HELLOWORLD" would be represented by the set of integers $m_1, m_2, \dots \{9, 6, 13, 13, 16, 24, 16, 19, 13, 5\}$

Using our table above, we obtain ciphertext integers $c_1, c_2, \dots \{3, 18, 19, 19, 4, 30, 4, 28, 19, 26\}$

Note that this example is no more secure than using a simple Caesar substitution cipher, but it serves to illustrate a simple example of the mechanics of RSA encryption.

Remember that calculating $m^e \bmod n$ is easy, but calculating the inverse $c^e \bmod n$ is very difficult, well, for large n 's anyway. However, if we can factor n into its prime factors p and q , the solution becomes easy again, even for large n 's. Obviously, if we can get hold of the secret exponent d , the solution is easy, too.

Key Generation Algorithm

1. Generate two large random primes, p and q , of approximately equal size such that their product $n = pq$ is of the required bit length, e.g. 1024 bits. [See note 1].

2. Compute $n = pq$ and $(\phi) \text{ phi} = (p-1)(q-1)$.
3. Choose an integer e , $1 < e < \text{phi}$, such that $\text{gcd}(e, \text{phi}) = 1$. [See note 2].
4. Compute the secret exponent d , $1 < d < \text{phi}$, such that $ed \equiv 1 \pmod{\text{phi}}$. [See note 3].
5. The public key is (n, e) and the private key is (n, d) . The values of p , q , and phi should also be kept secret.
 - n is known as the modulus.
 - e is known as the public exponent or encryption exponent.
 - d is known as the secret exponent or decryption exponent.

Encryption

Sender A does the following:-

1. Obtains the recipient B's public key (n, e) .
2. Represents the plaintext message as a positive integer m [see note 4].
3. Computes the ciphertext $c = m^e \pmod{n}$.
4. Sends the ciphertext c to B.

Decryption

Recipient B does the following:-

1. Uses his private key (n, d) to compute $m = c^d \pmod{n}$.
2. Extracts the plaintext from the integer representative m .

Source Code:

```
import java.util.Scanner;

public class RSA {
    static int gcd(int m, int n) {
        while (n != 0) {
            int r = m % n;
            m = n;
            n = r;
        }
        return m;
    }

    public static void main(String args[]) {
        int p, q, n, e = 0, d = 0, phi;
        int nummes[] = new int[100];
        int encrypted[] = new int[100];
```

```
int decrypted[] = new int[100];
int i, j, nofelem;
Scanner sc = new Scanner(System.in);
String message;
System.out.print("Enter the Message to be encrypted: ");
message = sc.nextLine();
System.out.println("Enter value of p and q:");
p = sc.nextInt();
q = sc.nextInt();
n = p * q;
phi = (p - 1) * (q - 1);
/* Find e */
for (i = 2; i < phi; i++) {
    if (gcd(i, phi) == 1) {
        e = i;
        break;
    }
}
/* Find d */
for (i = 2; i < phi; i++) {
    if ((e * i - 1) % phi == 0) {
        d = i;
        break;
    }
}
/* Convert message characters to numbers */
for (i = 0; i < message.length(); i++) {
    char c = message.charAt(i);
    if (c >= 'a' && c <= 'z') {
        nummes[i] = c - 'a' + 1;
    } else if (c >= 'A' && c <= 'Z') {
        nummes[i] = c - 'A' + 1;
    } else {
        nummes[i] = 0;
    }
}
```

```
}  
}  
nofelem = message.length();  
/* Encryption:  $C = M^e \bmod n$  */  
for (i = 0; i < nofelem; i++) {  
    encrypted[i] = 1;  
    for (j = 0; j < e; j++) {  
        encrypted[i] =  
            (encrypted[i] * nummes[i]) % n;  
    }  
}  
System.out.println("\nPublic Key (e, n) = (" + e + ", " + n + ")");  
System.out.println("Private Key (d, n) = (" + d + ", " + n + ")");  
  
System.out.println("\nEncrypted message:");  
for (i = 0; i < nofelem; i++) {  
    System.out.print(encrypted[i] + " ");  
}  
/* Decryption:  $M = C^d \bmod n$  */  
for (i = 0; i < nofelem; i++) {  
    decrypted[i] = 1;  
    for (j = 0; j < d; j++) {  
        decrypted[i] =  
            (decrypted[i] * encrypted[i]) % n;  
    }  
}  
System.out.println("\n\nDecrypted message:");  
for (i = 0; i < nofelem; i++) {  
    if (decrypted[i] == 0) {  
        System.out.print(" ");  
    } else {  
        System.out.print((char) (decrypted[i] + 'a' - 1));  
    }  
}  
}
```

```
        System.out.println();
        sc.close();
    }
}
```

Output1:

```
aiml@aiml-16:~$ gedit RSA.java
aiml@aiml-16:~$ javac RSA.java
aiml@aiml-16:~$ java RSA
Enter the Message to be encrypted: hello
Enter value of p and q:
5
7
```

Public Key (e, n) = (5, 35)

Private Key (d, n) = (5, 35)

Encrypted message:

8 10 17 17 15

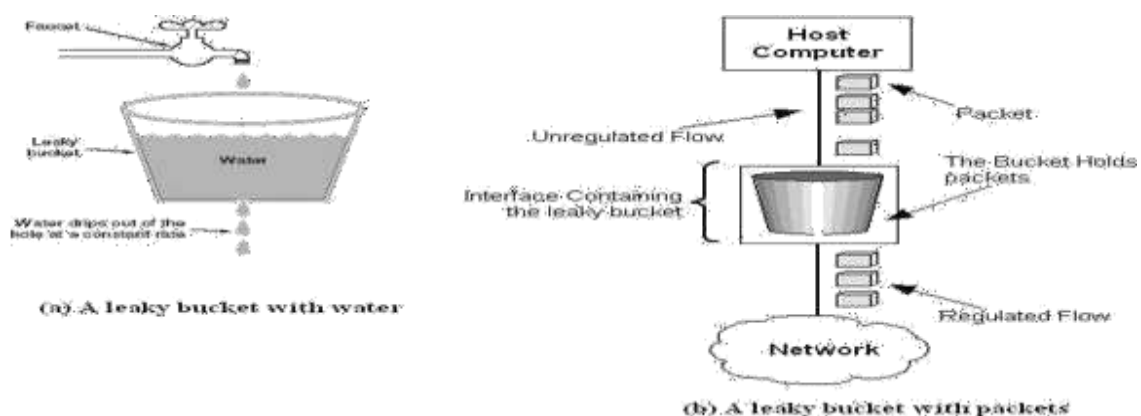
Decrypted message:

hello

Program 10: Write a program for congestion control using leaky bucket algorithm.**Program Objective:**

- Understand the operation of congestion control using leaky bucket algorithm.

The main concept of the leaky bucket algorithm is that the output data flow remains constant despite the variant input traffic, such as the water flow in a bucket with a small hole at the bottom. In case the bucket contains water (or packets) then the output flow follows a constant rate, while if the bucket is full any additional load will be lost because of spillover. In a similar way if the bucket is empty the output will be zero. From network perspective, leaky bucket consists of a finite queue (bucket) where all the incoming packets are stored in case there is space in the queue, otherwise the packets are discarded. In order to regulate the output flow, leaky bucket transmits one packet from the queue in a fixed time (e.g. at every clock tick). In the following figure we can notice the main rationale of leaky bucket algorithm, for both the two approaches (e.g. leaky bucket with water (a) and with packets (b)).



While leaky bucket eliminates completely bursty traffic by regulating the incoming data flow its main drawback is that it drops packets if the bucket is full. Also, it doesn't take into account the idle process of the sender which means that if the host doesn't transmit data for some time the bucket becomes empty without permitting the transmission of any packet.

Implementation Algorithm:

Steps:

1. Read The Data For Packets

2. Read The Queue Size
3. Divide the Data into Packets
4. Assign the random Propagation delays for each packets to input into the bucket (input_packet).
5. while((Clock++< total_packets))
 - a. if (clock == input_packet) i. insert into Queue
 - b. if (clock % 5 == 0) i. Remove packet from Queue
6. End

Source Code:

```
import java.util.Scanner;

public class leaky
{
    static int min(int x, int y)
    {
        if (x < y)
            return x;
        else
            return y;
    }

    public static void main(String[] args)
    {
        int drop = 0;
        int mini;
        int nsec;
        int cap;
        int count = 0;
        int i;
        int process;
        int inp[] = new int[25];
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the Bucket Size: ");
        cap = sc.nextInt();
        System.out.print("Enter the Operation Rate: ");
        process = sc.nextInt();
```

```
System.out.print("Enter the Number of Seconds You Want To Simulate: ");
nsec = sc.nextInt();
for (i = 0; i < nsec; i++)
{
    System.out.print("Enter the Size of the Packet Entering at "
        + (i + 1) + " sec: ");
    inp[i] = sc.nextInt();
}
System.out.println();
System.out.println("-----");
System.out.println("Second\tPacket Received\tPacket Sent\tPacket Left\tPacket Dropped");
System.out.println("-----");
for (i = 0; i < nsec; i++)
{
    drop = 0;
    count = count + inp[i];
    if (count > cap)
    {
        drop = count - cap;
        count = cap;
    }
    mini = min(count, process);
    count = count - mini;
    System.out.println((i + 1) + "\t"
        + inp[i] + "\t\t"
        + mini + "\t\t"
        + count + "\t\t"
        + drop);
}
/* Send remaining packets */
for (; count != 0; i++)
{
    drop = 0;
```

```
        mini = min(count, process);
        count = count - mini;
        System.out.println((i + 1) + "\t"
            + "0\t\t"
            + mini + "\t\t"
            + count + "\t\t"
            + drop);
    }
    sc.close();
}
}
```

Output1:

aiml@aiml-16:~\$ gedit leaky.java

aiml@aiml-16:~\$ javac leaky.java

aiml@aiml-16:~\$ java leaky

Enter the Bucket Size: 6

Enter the Operation Rate: 2

Enter the Number of Seconds You Want To Simulate: 2

Enter the Size of the Packet Entering at 1 sec: 8

Enter the Size of the Packet Entering at 2 sec: 6

Second	Packet Received	Packet Sent	Packet Left	Packet Dropped
1	8	2	4	2
2	6	2	4	4
3	0	2	2	0
4	0	2	0	0

Viva Questions and Answers

1. What is a Computer Network?

A collection of autonomous computing devices connected via transmission media to share hardware resources, data, and software applications.

2. Differentiate between OSI and TCP/IP models.

OSI has 7 distinct layers with strict boundaries and separate presentation/session layers. TCP/IP has 4 layers, combines presentation/session into the application layer, and is protocol-driven.

3. What is the function of the Physical Layer?

Transmits raw, unstructured bit streams over a physical communication medium (cables, wireless waves).

4. What is the role of the Data Link Layer?

Responsible for node-to-node data transfer, physical addressing (MAC), framing, and error detection.

5. What does the Network Layer do?

Handles logical addressing (IP addresses), packet forwarding, and route determination (routing) across multiple networks

6. What is the function of the Transport Layer?

Provides end-to-end reliable or unreliable delivery, flow control, and multiplexing using port numbers (e.g., TCP, UDP).

7. What is Encapsulation?

The process of adding protocol headers and trailers to data as it moves down the network stack from the application layer to the physical layer.

8. Why are headers and trailers added to frames?

Headers carry control information (source/destination addresses, protocol type) while trailers typically store error-checking data (like CRC bits).

9. What is MTU (Maximum Transmission Unit)?

The largest possible packet size (in bytes) that a given layer or medium can pass without needing fragmentation.

10. Differentiate between Flow Control and Congestion Control.

Flow control manages the rate of data transmission between a single sender and receiver to prevent buffer overflow. Congestion control prevents the entire network from collapsing due to over-subscription of intermediate router resources.

11. Differentiate between TCP and UDP.

TCP is connection-oriented, reliable, and provides error-recovery and acknowledgments. UDP is connectionless, faster, and provides no delivery guarantees.

12. What is an IP Address?

A unique logical 32-bit (IPv4) or 128-bit (IPv6) identifier assigned to a device participating in a computer network.

13. What is a MAC Address?

A 48-bit physical hardware address permanently assigned to a Network Interface Card (NIC) by the manufacturer.

4. What is the 3-Way Handshake in TCP?

The connection establishment sequence: **SYN** → **SYN-ACK** → **ACK**

15. What is ARP (Address Resolution Protocol)?

A protocol used to map a known logical IPv4 address to its corresponding physical MAC address on a local area network.

16. What is RARP?

Reverse Address Resolution Protocol; allows a physical machine to discover its logical IP address from a gateway router when it boots up.

17. What is a Port Number?

A 16-bit integer identifier (0–65535) used by the transport layer to route incoming packets to the correct application process.

18. What is the difference between IPv4 and IPv6?

IPv4 uses 32-bit addresses and dotted-decimal notation. IPv6 uses 128-bit addresses, hexadecimal notation, built-in security (IPsec), and larger address spaces.

19. What is DNS (Domain Name System)?

A hierarchical distributed database that translates human-readable hostnames (like vtu.ac.in) into numerical IP addresses.

20. Which transport layer protocol does DNS use?

It uses **UDP** (port 53) for fast queries/responses and **TCP** (port 53) for zone transfers between servers.

21. What is a Hub?

A physical layer broadcast device that repeats incoming electrical signals out to all other connected ports without filtering.

22. What is a Switch?

A data-link layer device that uses MAC address tables to intelligently forward frames only to the designated destination port.

23. What is a Router?

A network layer device that connects distinct logical networks and routes packets based on IP destination headers.

24. What is a Gateway?

An internetworking device or node that acts as a translator between completely different network architectures or protocol suites.

25. What is a Bridge?

A data-link layer hardware device that connects two separate Local Area Network (LAN) segments to reduce traffic congestion.

26. What is a Repeater?

A physical layer device that receives a weakened signal, amplifies/regenerates it, and retransmits it to extend maximum cable distances.

27. What is a Firewall?

A network security system that filters incoming and outgoing network traffic based on predetermined organizational security rules.

28. Define Mesh Topology.

A network setup where every single node has a dedicated, point-to-point physical connection to every other node.

29. What is Star Topology?

A layout where all remote nodes connect directly to a central hub or switch.

30. What is Bus Topology?

A network structure where all nodes are tied directly to a single central communication line (the backbone cable).

31. What is NS-2 (Network Simulator 2)?

An open-source, discrete event-driven simulation tool heavily used to model wired and wireless routing, queue behaviors, and transport protocols.

32. Which languages are used to write NS2 simulation programs?

OTcl (Object Tool Command Language) for setting up network topologies/scenarios, and **C++** for implementing internal protocol mechanics.

33. What does NAM stand for in NS2?

Network Animator, a visual animation tool that plays back the trace file generated during an NS2 simulation run.

34. What is a Trace File (.tr) in NS2?

An ASCII text log file that records every single network event (enqueue, dequeue, receive, drop) occurring across simulation links.

35. What do the symbols r, +, -, and d mean in an NS2 trace file?

- r: Packet received at a node/link endpoint.
- t or +: Packet enqueued into the link buffer.
- -: Packet dequeued from the buffer.
- d: Packet dropped due to buffer overflow/congestion.

36. What is the purpose of the tcpdump command?

A command-line utility used to capture, intercept, and inspect live TCP/IP packets flowing across a system network interface

37. What is traceroute used for?

A diagnostic tool that tracks the exact router path packets take to reach a target destination by varying TTL (Time To Live) values.

38. What is a Socket?

An endpoint for bidirectional communication running between two separate programs across a computer network (combination of an IP address and a Port number).

39. What is the difference between TCP Sockets and Datagram (UDP) Sockets?

TCP sockets use stream connections (SOCK_STREAM) requiring active server connection acceptances. UDP sockets use datagrams (SOCK_DGRAM) sending independent packets without tracking persistent connections.

40. What is the function of the bind() system call in socket programming?

Associates a specific local IP address and port number configuration profile directly with a created socket descriptor.

41. What is CRC (Cyclic Redundancy Check)?

An error-detecting code technique that appends a fixed-size check value (remainder of polynomial division) to data frames.

42. What is a Generator Polynomial in CRC?

A predefined bit pattern utilized as the divisor during binary polynomial division to calculate frame check sequences (e.g., CRC-CCITT).

43. Explain Distance Vector Routing (DVR).

A dynamic routing algorithm (like RIP) where routers periodically share their entire routing table metrics with immediate neighbors using the Bellman-Ford equation.

44. What is the "Count-to-Infinity" problem in DVR?

A routing loop scenario where slow propagation of link metric increases causes routers to continuously increment hop counts infinitely.

45. Explain Link State Routing (LSR).

An algorithm (like OSPF) where every node broadcasts its local link conditions to the entire network topology map, computing shortest paths using Dijkstra's algorithm.

46. What is the Leaky Bucket Algorithm?

A congestion control mechanism that polices and shapes bursty network traffic into a smooth, uniform output flow rate.

47. What is the Token Bucket Algorithm?

A congestion control algorithm that permits burst transmissions up to a preset capacity limit as long as accumulated system "tokens" are available.

48. What is RSA Algorithm?

An asymmetric public-key cryptography algorithm used for secure data encryption and decryption utilizing large prime number factorization.

49. What is the difference between HTTP and HTTPS?

HTTP transmits raw plaintext data over port 80. HTTPS secures data exchanges using TLS/SSL encryption protocols over port 443.

50. What is the purpose of the netstat command?

Displays active network connections, listening ports, routing tables, and interface statistics.